UNIT -2

M. Mounika (192411083)

CSA1006-SOFTWARE ENGINEERING

Problem 3

Choosing Architecture for a Scalable Food Delivery Platform

Scenario

A startup wants to build a food delivery platform similar to **Swiggy** or **Zomato**. The platform should handle thousands of customer orders during peak hours (lunch and dinner times). The CTO needs to decide whether to use **Monolithic**, **Microservices**, or **Serverless Architecture**.

Task 1: Recommended Architecture Choice

Recommended Architecture: Microservices Architecture

Explanation

Microservices Architecture is the most suitable choice for a food delivery platform because the application consists of many independent functions such as user management, restaurant management, order processing, payment, delivery tracking, and notifications.

Instead of developing everything as one large application, the system is divided into small independent services. Each service performs one specific task and communicates with other services using APIs or events.

This architecture provides high scalability, flexibility, and reliability, making it ideal for applications that receive thousands of requests every minute.

Task 2: Three Technical Reasons for Choosing Microservices

1. High Scalability

During lunch and dinner hours, the number of food orders increases rapidly.

With Microservices:

- Only the **Order Service** can be scaled if order traffic increases.
- Payment or Delivery services remain unchanged.

Example

If 20,000 users place orders at the same time:

- Increase only Order Service servers.
- Save infrastructure cost.

2. Independent Deployment

Each service is developed and deployed separately.

For example:

- Payment team updates the payment system.
- Delivery team updates delivery tracking.

These updates do not stop the complete application.

Benefit

- Faster software updates
- Less downtime
- Easier maintenance

3. Better Fault Isolation

Suppose the Payment Service crashes.

In a monolithic system:

- Entire application may stop working.

In Microservices:

- Only payment service is affected.
- Customers can still browse restaurants and menus.

This improves system reliability.

Task 3: Role of Event-Driven Architecture

What is Event-Driven Architecture?

Event-Driven Architecture is a communication model where one service generates an **event**, and other services automatically respond to that event.

Instead of waiting for direct communication, services exchange events using a message broker like **Kafka** or **RabbitMQ**.

Example

Customer places an order.

↓

Order Service creates:

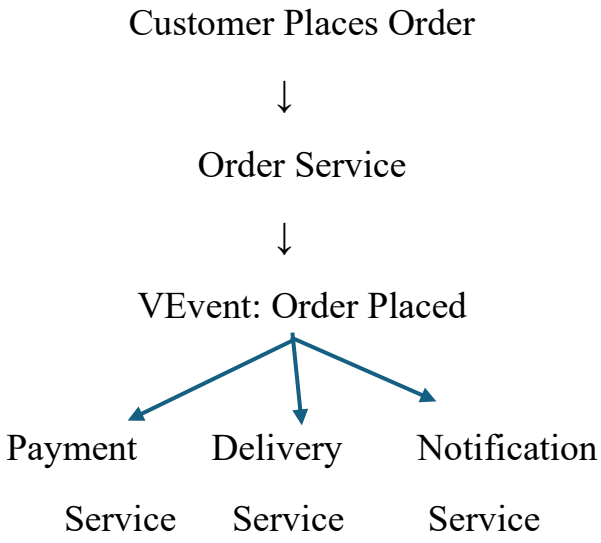
Event → "Order Placed"

This event is received by:

- Payment Service
- Delivery Service
- Notification Service
- Restaurant Service

Each service performs its work independently.

Workflow Diagram



Benefits of Event-Driven Architecture

1. Faster Processing

Multiple services work simultaneously.

2. Loose Coupling

Services do not depend directly on each other.

3. Easy Scalability

Each service can scale independently.

4. Better User Experience

Customers receive real-time notifications about their orders.

Task 4: Suitable Microservices

1. Order Service

Responsibilities

- Accept customer orders
- Validate order details
- Calculate total bill
- Store order information

Input

- Customer ID
- Restaurant ID
- Food Items

Output

- Order Confirmation

2. Payment Service

Responsibilities

- Process online payments
- Verify payment
- Generate payment receipt

Input

- Order ID
- Payment Method

Output

- Payment Success or Failure

3. Delivery Service

Responsibilities

- Assign delivery partner
- Track delivery location

- Update delivery status

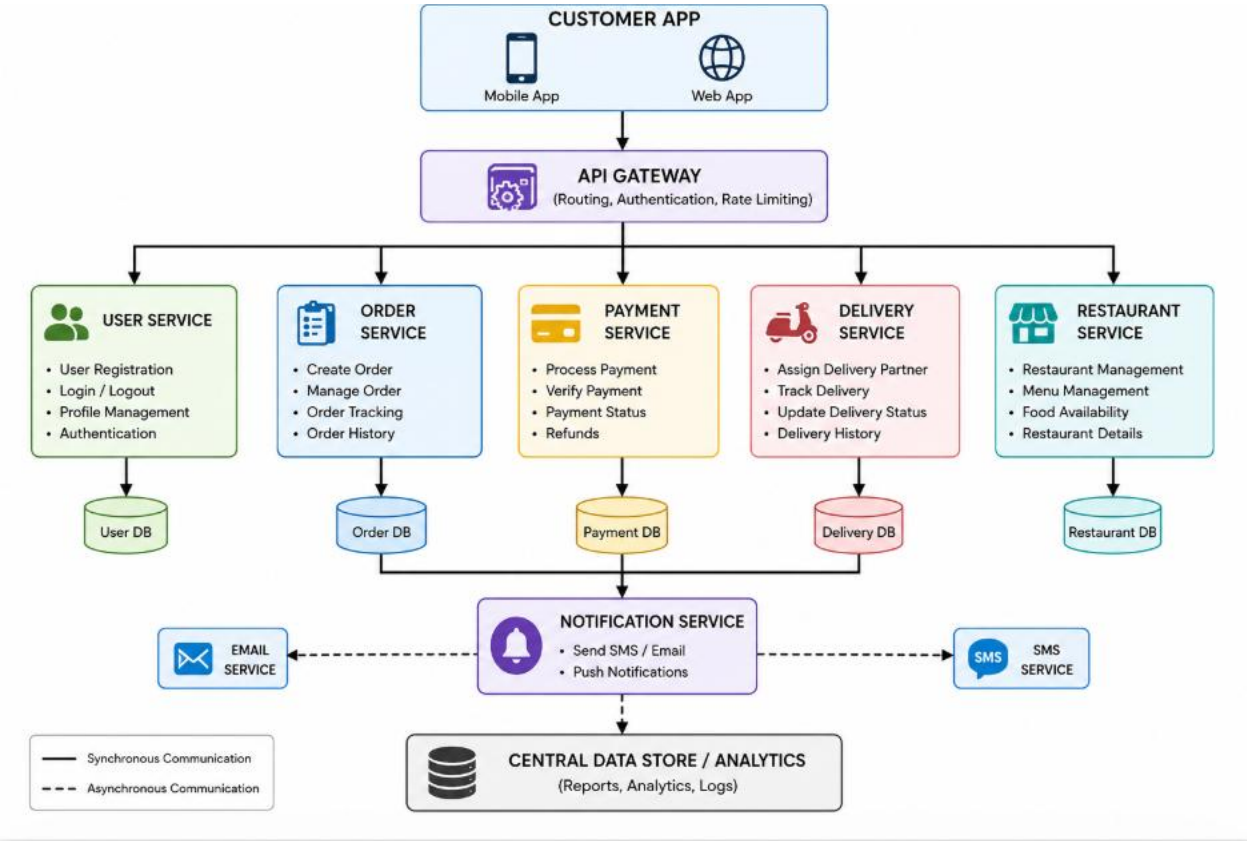
Input

- Order ID
- Delivery Address

Output

- Delivery Status

Architecture Diagram



Advantages of Microservices

- High scalability
- Faster deployment
- Better fault isolation
- Easy maintenance

- Independent development
- Supports cloud deployment
- Improved system performance
- Better reliability

Conclusion

For a large-scale food delivery platform like **Swiggy** or **Zomato**, **Microservices Architecture** is the best choice because it supports high scalability, independent deployment, fault isolation, and faster development. When combined with **Event-Driven Architecture**, services can communicate efficiently, process thousands of orders simultaneously, and provide real-time updates to customers. By dividing the system into independent services such as **Order Service**, **Payment Service**, and **Delivery Service**, the platform becomes more reliable, flexible, and capable of handling peak-hour traffic efficiently.